Meshtastic LoRa Room Server

Soutenance finale · Projet Semestre S8

Objectif : ajouter une couche applicative de salons persistants au-dessus du réseau mesh Meshtastic, avec une architecture légère, robuste et adaptée aux contraintes LoRa.

INFO4 2025-2026 | Polytech | Encadrant : Francois Letellier

Équipe

TAWFIK Adam
UNAY Arhan

Mars 2026

Contexte et besoin

Pourquoi ce projet est utile dans un réseau LoRa mesh

Constat initial

Meshtastic permet surtout de la messagerie texte diffuse. En revanche, il ne fournit pas nativement de salons persistants, d'historique structuré ni d'organisation des échanges.

Besoin identifié

Permettre à des utilisateurs de créer des rooms, publier des messages et relire un historique, même après redémarrage du serveur.

Contexte d'usage

Scénarios à infrastructure limitée : terrain, zones isolées, environnement dégradé ou besoin d'un service compact autonome.

Diffusion simple → **Besoin de structure** → **Room Server**

- Ajouter une couche applicative sans modifier le protocole Meshtastic.
- Rester compatible avec du matériel standard : Raspberry Pi 3B + Wio-E5 Mini.
- Conserver une solution légère, démontrable et robuste.

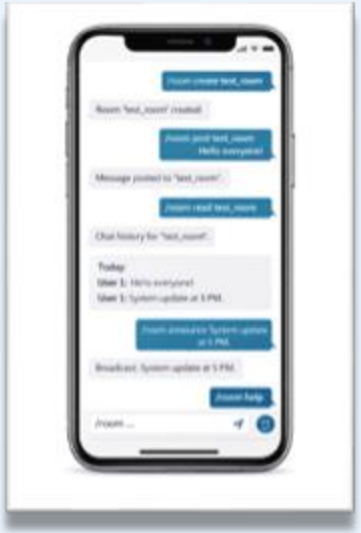
Objectifs et périmètre fonctionnel

Ce que le système devait faire pendant le semestre

Objectifs principaux

- Interface par commandes
- Persistance locale
- Déploiement sur Raspberry Pi
- Robustesse après reboot
- Démonstrateur réel + documentation

Le choix a été de viser un noyau fonctionnel fiable avant toute extension.



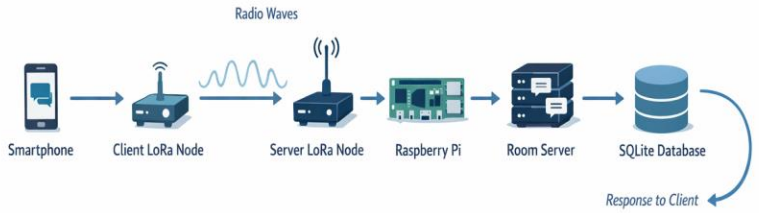
Fonctionnalités réalisées

Create room	**Implémenté**	**Création d'une room par nom unique**
Annonce Messages	**Implémenté**	**Diffusion d'un message global d'annonce à toutes les rooms / tous les utilisateurs selon votre logique serveur**
List rooms	**Implémenté**	**Retour de la liste des rooms stockées**
Post message	**Implémenté**	**Ajout d'un message dans une room**
Read history	**Implémenté**	**Lecture des N derniers messages avec limites**
Help command	**Implémenté**	**Résumé concis des commandes**
Persistence after reboot	**Implémenté**	**Données conservées après redémarrage**
Room info	**Implémenté**	**Affichage des informations d'une room (nom, description, métadonnées utiles)**

Architecture de la solution

Chaîne complète : client LoRa → Room Server → base SQLite

Smartphone / client → **Noeud LoRa client** → **Noeud serveur Wio-E5** → **Raspberry Pi 3B** → **Application Python Room Server**



Meshtastic serial interface → **Parser + RoomManager** → **SQLite**

- Architecture modulaire : transport, parsing, logique métier, persistance.
- Pas de modification du protocole radio : la valeur est ajoutée côté application.
- Déploiement réel sur matériel embarqué.

Fonctionnement logiciel

Modules principaux et rôle de chaque composant

main.py	Orchestrateur : initialise DB, RoomManager et connexion matérielle
parser.py	Analyse la syntaxe des commandes /room et extrait les arguments
room_manager.py	Cœur métier : rooms, anti-spam, formatage, lecture et écriture
database.py	Stockage SQLite : tables rooms/messages et opérations CRUD
meshtastic_comm_hw.py	Interface réelle avec le module LoRa via /dev/ttyUSB*
client.py	Client de test pour envoyer les commandes au serveur
logger.py / reset_db.py	Supervision, debugging et maintenance

Structure choisie

Chaque module a une responsabilité claire. Cela facilite les tests, le debug et la maintenance.

Persistance

SQLite a été retenu pour ses transactions atomiques et sa meilleure robustesse qu'un simple fichier JSON.

Contributions

Adam : parsing, hardware comm, client, orchestration.
Arhan : DB, room management, simulation, logging, maintenance.

Contraintes LoRa et choix techniques

Les compromis qui ont guidé l'implémentation

Payload limité

Les réponses doivent rester courtes pour éviter la saturation radio et limiter les pertes.

Faible bande passante

La lecture d'historique est volontairement limitée et les réponses longues sont découpées.

Interférences réseau

Utilisation d'un canal privé avec PSK dédiée pour isoler les tests.

Réponses apportées

- Commandes courtes : /room create, /room list, /room post, /room read,/room info, /room annonce
- Chunking des messages trop longs pour améliorer la lisibilité et réduire le risque de perte.
- Messages administratifs envoyés en privé quand c'est pertinent.
- systemd pour redémarrage automatique au boot ou après échec.
- Surveillance du matériel et stratégie de reprise après déconnexion USB.

Idée clé : dans un environnement contraint, la valeur vient moins de la complexité que du bon compromis entre simplicité, robustesse et démontrabilité.

Validation et démonstration

Ce qui a été effectivement testé sur le démonstrateur

Créer une room	Commande exécutée et accusé de réception	Validé
Poster un message	Message stocké dans la bonne room	Validé
Lire l'historique	Retour des derniers messages formatés	Validé
Redémarrer le serveur	Données toujours présentes après reboot	Validé
Débrancher le module USB	Sortie propre puis relance via systemd	Validé
Canal privé	Isolation par rapport au trafic public	Validé

Stratégie

Validation progressive : d'abord en environnement mock, puis sur matériel réel.

Résultat principal

Le projet a démontré qu'un nœud Raspberry Pi peut étendre Meshtastic avec des rooms persistantes et une reprise contrôlée après panne.

Limite assumée

La campagne est surtout qualitative : peu de mesures massives de latence multi-clients.

Bilan, apport et perspectives

Ce que le projet prouve et ce qu'il reste à faire

Bilan

Le cœur fonctionnel demandé a été réalisé : rooms, messages persistants, lecture d'historique, robustesse après redémarrage et déploiement embarqué.

Apport

Le Room Server apporte une organisation structurée des échanges au-dessus d'un réseau initialement orienté diffusion simple.

Perspectives d'évolution

- Gestion des droits et rooms privées.
- Interface web locale hébergée sur le Raspberry Pi.
- Synchronisation entre plusieurs Room Servers.
- Tableau de bord de supervision et statistiques d'usage.
- Mesures quantitatives plus poussées sous charge contrôlée.

Merci pour votre attention 😉